

目录

第一章 设计理念与架构映射总览	3
1.1 核心设计原则	4
第二章 三层架构设计	4
2.1 StudioDaemon 层（借鉴 apps/daemon）	5
2.2 StudioWeb 层（借鉴 apps/web）	5
2.3 Agent Execution 层（借鉴 CLI spawn 模式）	5
第三章 多模态 Agent 适配器体系	6
3.1 MultimodalAgentDef 接口（借鉴 RuntimeAgentDef）	6
3.2 四大模态适配器族	6
3.3 AG-UI 双向适配器（借鉴 packages/agui-adapter）	7
第四章 多模态提示词组合系统	7
4.1 21 层提示词优先级栈	7
4.2 新增层详解：媒体创作上下文	8
4.3 新增层详解：跨模态一致性指令	8
第五章 技能即文件系统	9
5.1 SKILL.md 多模态扩展（借鉴 SKILL.md 格式）	9
5.2 studio.json Schema（借鉴 open-design.json + Zod）	9
第六章 多模态流式协议	9
6.1 media:chunk 帧结构（借鉴 SseTransportEvent 信封格式）	10
第七章 多工件流式渲染	10
第八章 多模态评审剧院	11
8.1 Rolling Ratchet 按模态独立发布（借鉴 evaluateRollout）	12
第九章 Contracts 共享缝合层	12
第十章 Sidecar IPC 协作	13
第十一章 能力门控技能系统	13

第十二章 品牌系统即 Markdown	14
第十三章 本地优先与 BYOK 模式	14
第十四章 完整数据流与包结构	15
14.1 Monorepo 包结构（借鉴 Open Design monorepo 组织）	16
14.2 跨模式依赖链	16

第一章 设计理念与架构映射总览

Open Design 项目展现了一种高度成熟的 AI 原生应用架构范式：将 Agent 视为可替换的 CLI 工具，将插件视为纯文件系统文件夹，将设计系统表达为 Markdown，通过 Contracts 层缝合前后端类型，通过 SSE 流式协议统一多 Agent 输出，通过 Critique Theater 实现渐进式质量保证。这些模式并非偶然的设计选择，而是经过大规模生产验证的架构决策，每一个模式都解决了特定的工程痛点。

AI Agent Studio 的核心设计目标，是在 Open Design 已验证的架构模式基础上，将其从单一的 Web 设计领域扩展到文字、图片、视频、音乐四大内容创作模态。这并非简单的功能叠加，而是需要对每个架构模式进行多模态适配：Agent 适配器需要理解不同媒体的输入输出格式，流式渲染需要支持渐进式图片加载和音频波形绘制，评审剧院需要为每种模态设计专属的评审维度，提示词组合系统需要注入多模态上下文信息。

下表展示了 Open Design 的每一个核心架构模式如何被精确映射到 AI Agent Studio 的多模态创作场景中，每一行都标注了原始模式的实现位置、关键代码接口，以及在 Agent Studio 中的对应设计。这种逐模式映射确保了没有任何一个架构模式被遗漏或浅尝辄止。

Open Design 模式	原始实现关键接口	Agent Studio 对应设计
Agent-agnostic Adapter (22+ CLI 适配器)	RuntimeAgentDef buildArgs() + streamFormat	MultimodalAgentDef 4 大模态适配器族 buildArgs() + mediaStreamFormat
3-Layer Architecture (Daemon/Web/CLI)	Express 5 + SQLite Next.js + CLI spawn	StudioDaemon + StudioWeb + AgentProcess 同一分层，扩展多模态路由
14+ Layer Prompt Composition	composeSystemPrompt() 严格优先级栈	composeMultimodalPrompt() 21 层提示词栈 新增 7 个模态专属层
Plugin-as-Filesystem (SKILL.md + JSON)	SKILL.md + open-design.json 零构建步骤	SKILL.md + studio.json 多模态技能声明式定义 零构建步骤
SSE Streaming Protocol (6 事件类型)	ChatSseEvent start/agent/stdout/end	MultimodalSseEvent 12+ 事件类型 含 media:chunk/progress/complete
Artifact Streaming Rendering	标签解析 srcdoc iframe 沙箱	MultiArtifactParser 4 种媒体渲染器 沙箱化预览
Critique Theater (5 角色评审)	designer/critic/brand /ally/copy + Rolling Ratchet	MultimodalCritique 6 角色 + modalist 按模态独立发布轨道
Contracts as Architecture Seam	packages/contracts Zod + zero deps	packages/studio-contracts Zod + zero deps 扩展多模态类型
Sidecar IPC (Unix Socket)	requestJsonIpc() JSON-over-Unix-Socket	requestJsonIpc() + streamMediaIpc() 新增二进制流传输

Capability-gated Plugins	od.capabilities trust tier + runtime check	studio.capabilities 扩展媒体能力词汇 trust tier + runtime check
Design System as Markdown	DESIGN.md 9 节品牌规范	BRAND.md 12 节多模态品牌规范 含音频/视频/图片风格
Local-first + BYOK	CLI 本地模式 API 代理 + SSRF 防护	本地 Studio 模式 BYOK 多模态 API 代理 SSRF + 媒体配额

表 1-1 Open Design 到 AI Agent Studio 的架构模式映射

1.1 核心设计原则

- **Agent 不可知原则**：平台不绑定任何特定 AI 模型或服务。所有 Agent 通过统一的 MultimodalAgentDef 接口接入，平台只需知道如何构建 CLI 参数和解析输出流。新增一个 Agent 只需实现一个适配器对象，无需修改平台核心代码。这一原则直接借鉴 Open Design 的 22 个 CLI 适配器设计。
- **文件即插件原则**：技能和插件以纯文件系统文件夹的形式存在，包含 SKILL.md 和 studio.json。没有构建步骤，没有编译过程。复制文件夹即安装插件，删除文件夹即卸载插件。这一原则直接借鉴 Open Design 的 Plugin-as-Filesystem 模式。
- **合约缝合原则**：所有跨层共享的类型定义都放在零依赖的 studio-contracts 包中，使用 Zod Schema 定义运行时校验规则。Daemon 层和 Web 层都依赖这个包，而非直接相互引用。这一原则直接借鉴 Open Design 的 packages/contracts 设计。
- **流式优先原则**：所有多模态内容生成过程都通过 SSE 流式协议传输，前端可以实时渲染部分结果。文字流逐字输出，图片流渐进式解码，音频流分帧播放，视频流逐帧预览。这一原则直接借鉴 Open Design 的 SSE Streaming Protocol 设计。
- **能力门控原则**：每个技能声明的所需能力在运行时被严格检查。受信任的技能自动获得更多能力，不受信任的技能只能使用最基础的功能。这一原则直接借鉴 Open Design 的 Capability-gated Plugins 设计。
- **渐进式质量保证原则**：借鉴 Critique Theater 的 Rolling Ratchet 机制，新功能从 M0 逐步发布到 M3，每一步都需要达到足够的质量指标才能晋级。评审不是阻碍创新的门神，而是确保交付质量的护栏。

第二章 三层架构设计

Open Design 的三层架构（Daemon/Web/CLI Execution）是其最核心的架构决策之一。这种分层将持久化和路由逻辑放在 Daemon 层，将 UI 渲染和交互放在 Web 层，将 AI 推理执行放在独立的 CLI 进程中。三层之间通过 HTTP/SSE（Daemon-Web）和 stdin/stdout（Daemon-CLI）通信，实现了关注点分离和进程隔离。AI Agent Studio 完全继承这一架构，

并针对多模态场景进行了必要的扩展。

2.1 StudioDaemon 层（借鉴 apps/daemon）

StudioDaemon 是整个平台的中枢，直接对应 Open Design 的 apps/daemon。它基于 Express 5 构建，使用 SQLite（WAL 模式）作为本地数据库——这与 Open Design 的 db.ts 完全一致，同样使用 better-sqlite3，同样执行 PRAGMA journal_mode = WAL 和 PRAGMA foreign_keys = ON。关键扩展点在于多模态资产存储：Open Design 只需要存储 HTML 片段和元数据，而 Agent Studio 需要管理图片、音频和视频文件，这些文件需要比 HTML 片段更大的存储空间和更精细的元数据管理。

数据库设计在 Open Design 原有 projects、conversations、messages、agent_sessions 等表的基础上新增了 multimodal_assets 表（存储生成内容的类型、路径、分辨率、时长、缩略图等信息）和 asset_generations 表（记录每次生成的参数、使用的 Agent ID、耗时、质量评分等追踪信息）。这种扩展策略遵循了 Open Design 的核心原则：不修改原有表结构，只追加新表。

2.2 StudioWeb 层（借鉴 apps/web）

StudioWeb 直接对应 Open Design 的 apps/web，基于 Next.js 构建，采用 Catch-All Routing 模式。SSE Provider 和 Daemon 连接 Provider 与 Open Design 完全一致。核心变化在于 Artifact 渲染系统：Open Design 只需要将 HTML/CSS 代码渲染到 srcdoc iframe 中（通过 apps/web/src/artifacts/parser.ts 和 apps/web/src/runtime/srcdoc.ts），而 Agent Studio 需要同时支持文字预览、图片渲染（含渐进式加载）、音频波形可视化与播放器、视频帧预览与播放控制。

2.3 Agent Execution 层（借鉴 CLI spawn 模式）

Agent Execution 层直接借鉴 Open Design 的 CLI spawn 模式。Open Design 通过 RuntimeAgentDef.buildArgs() 构建 CLI 参数数组，然后 spawn 子进程执行，通过 stdin 传递提示词，通过 stdout 读取流式输出。Agent Studio 完全继承这一模式，核心扩展在于引入了 mediaStreamFormat 字段：text-stream（与 Open Design 的 stream-json 兼容）、image-stream（Base64 分片 + 渐进式 JPEG）、audio-stream（PCM/MP3 分帧）、video-stream（关键帧 + 增量帧）。Daemon 层根据 MultimodalAgentDef 中声明的 mediaStreamFormat 选择对应的流式解析器。

第三章 多模态 Agent 适配器体系

Open Design 最精妙的架构决策之一是 Agent-agnostic Adapter Pattern。22 个 AI Agent CLI 适配器共享同一个 RuntimeAgentDef 接口，通过 buildArgs() 方法将用户提示词转换为特定 Agent 的 CLI 参数，通过 streamFormat 字段告诉 Daemon 如何解析 Agent 的输出流。Agent Studio 将这一模式扩展到多模态领域，设计了 MultimodalAgentDef 接口，同时保留了对全部 22 个文本 Agent 适配器的兼容性。

3.1 MultimodalAgentDef 接口（借鉴 RuntimeAgentDef）

MultimodalAgentDef 在 Open Design 的 RuntimeAgentDef 基础上扩展了多模态相关字段。RuntimeAgentDef 的核心接口包括 id、name、bin、buildArgs()、streamFormat、promptViaFile、promptViaStdin、externalMcpInjection、resumesSessionViaCli、capabilityFlags 等字段。MultimodalAgentDef 保留了所有这些字段，新增了 mediaStreamFormat（媒体流格式）、supportedMedia（支持的媒体类型数组）、imagePaths/audioPaths/videoPaths（媒体输入路径构建函数）。这种扩展方式确保了任何符合 RuntimeAgentDef 规范的适配器都可以直接作为 MultimodalAgentDef 使用——它只是 supportedMedia 为 ["text"] 的特例。

字段名	类型	来源	说明
id	string	继承	Agent 唯一标识，如 "claude"、"gpt-image"
buildArgs()	function	继承	构建 CLI 参数数组，新增 mediaType 参数
streamFormat	string	继承	文本流格式："stream-json" "claude-stream-json"
mediaStreamFormat	string	新增	媒体流格式："image-stream" "audio-stream" "video-stream"
supportedMedia	array	新增	支持的媒体类型：["text","image","audio","video"]
imagePaths	function	新增	图片输入路径构建，用于多模态理解
audioPaths	function	新增	音频输入路径构建
promptViaStdin	boolean	继承	是否通过标准输入传递提示词
externalMcpInjection	string	继承	MCP 注入方式
capabilityFlags	object	继承	运行时探测的能力标志

表 3-1 MultimodalAgentDef 接口关键字段

3.2 四大模态适配器族

Agent Studio 将适配器分为四大模态族，每个族共享相似的媒体流格式和输入输出模式。这种族的设计灵感来自 Open Design 中 Claude 和 Codex 适配器的相似性——它们虽然输出格式不同，但都是文本优先的 Agent。文字适配器族直接继承 Open Design 的全部 22 个文本 Agent 适配器（Claude、Codex、Gemini、Devin、Grok、Aider 等），无需任何修改。图片适配器族新增 DALL-E CLI、Stable Diffusion CLI、Midjourney Proxy CLI、FLUX CLI、Ideogram CLI 等，mediaStreamFormat 设为 image-stream。音频适配器族新增 Suno CLI、Udio CLI、ElevenLabs TTS CLI、MusicGen CLI 等，mediaStreamFormat 设为 audio-stream。视频适配器族新增 Runway CLI、Pika CLI、Sora Proxy CLI、Kling CLI 等，mediaStreamFormat 设为 video-stream。

3.3 AG-UI 双向适配器（借鉴 packages/agui-adapter）

Open Design 的 AG-UI 适配器（packages/agui-adapter）实现了 Open Design 原生事件与 AG-UI 规范事件之间的双向转换。encodeOdEventForAgui() 函数将 OD 原生事件（message_chunk、tool_call、end、genui_surface_request 等）映射到 AG-UI 规范事件（agent.message、tool_call、run.lifecycle 等）。Agent Studio 完全继承这一机制，并扩展了多模态事件的映射：image_chunk 映射到 AG-UI 的 tool_call 事件，audio_frame 和 video_segment 映射到 AG-UI 的 state_update 事件。这种设计确保了 Agent Studio 可以与任何 AG-UI 兼容的前端框架无缝集成。

第四章 多模态提示词组合系统

Open Design 的 composeSystemPrompt() 函数通过 14+ 层的严格优先级栈组合系统提示词。每一层都有明确的优先级和作用域，高优先级层可以覆盖低优先级层的指令。Agent Studio 的 composeMultimodalPrompt() 在此基础上扩展至 21 层，新增 7 个多模态专用层，这些层精确插入在优先级栈的合适位置，确保多模态上下文不会干扰核心身份和基础 workflow。

4.1 21 层提示词优先级栈

优先级	层名称	类型	说明
1	提示词注入阻力	继承	PROMPT_INJECTION_RESISTANCE 常量
2	API 模式覆盖	继承	streamFormat === "plain" 时简化输出
3	聊天模式覆盖	继承	sessionMode === "chat"
4-6	示例/发现/语言覆盖	继承	与 Open Design 完全一致
7	发现与哲学	继承	DISCOVERY_AND_PHILOSOPHY

8	基础身份章程	继承	BASE_SYSTEM_PROMPT
9	个人记忆	继承	memoryBody: 用户偏好事实
10-11	用户/项目自定义指令	继承	与 Open Design 完全一致
12	品牌系统 BRAND.md	扩展	替代 DESIGN.md, 12 节多模态品牌规范
13	媒体创作上下文	新增	mediaContext: 目标媒体类型、格式约束、创作阶段
14	多模态风格指南	新增	multimodalStyleGuide: 视觉/听觉/叙事风格指令
15	活跃技能 SKILL.md	继承	skillBody + skillName + 预检规则
16-17	插件块/活跃阶段块	继承	与 Open Design 完全一致
18	项目元数据	继承	metadata: 类型、平台、保真度
19	多模态输出合约	新增	mediaOutputContract: 分辨率、时长、码率
20	跨模态一致性指令	新增	crossModalConsistency: 多模态间风格和主题约束
21	Critique Theater 协议	继承	critique 配置 + renderPanelPrompt()

表 4-1 21 层提示词优先级栈

4.2 新增层详解：媒体创作上下文

第 13 层 mediaContext 为 Agent 提供当前创作任务的完整上下文信息，这是 Agent Studio 相对于 Open Design 的核心增量。Open Design 只需要处理 HTML/CSS 的输出格式，所有 Agent 的输出本质上都是文本流。而 Agent Studio 需要为每种媒体类型定义不同的输出规范：图片生成需要指定分辨率、宽高比、格式和风格参考；音频生成需要指定时长、采样率、BPM 范围和情绪标签；视频生成需要指定分辨率、帧率、时长和运镜偏好。mediaContext 层将这些信息结构化注入到系统提示词中，确保 Agent 的输出符合创作任务的格式要求。

4.3 新增层详解：跨模态一致性指令

第 20 层 crossModalConsistency 是多模态创作中最独特的层。当任务涉及多种媒体类型时（例如同时生成视频和配乐），这一层注入一致性约束指令，确保：视觉风格与音乐情绪匹配、视频节奏与音乐节拍同步、文案内容与画面叙事一致、品牌色彩在所有媒体中统一。这一层的存在使得 Agent Studio 不同于简单的多 Agent 并行调用——它通过提示词层面的约束实现了真正的多模态协同创作。这是 Open Design 中不存在的全新维度，因为 Open Design 的所有创作输出都是同一模态（HTML/CSS）。

第五章 技能即文件系统

Open Design 的 Plugin-as-Filesystem 模式是其最具创新性的设计之一。每个插件就是一个文件夹，包含 SKILL.md（YAML Frontmatter + Markdown workflow 描述）和 open-design.json（Zod Schema 验证的元数据清单）。Daemon 通过文件系统扫描发现插件（apps/daemon/src/plugins/registry.ts 的 resolvePluginFolder），通过 Zod safeParse 验证清单（packages/plugin-runtime/src/parsers/manifest.ts 的 parseManifest），通过 plugin-runtime 包的合并策略合并多个清单来源（mergeManifests）。Agent Studio 完全继承这一模式，将 open-design.json 重命名为 studio.json，将 SKILL.md 扩展为支持多模态技能声明。

5.1 SKILL.md 多模态扩展（借鉴 SKILL.md 格式）

SKILL.md 的 YAML Frontmatter 在 Open Design 基础上新增了 mediaTypes 字段和 mediaInputs 字段。mediaTypes 声明技能支持的媒体类型（如 [audio, text]），mediaInputs 定义每种媒体类型的输入参数。Markdown 正文部分则描述多模态 workflow 步骤。这种扩展方式与 Open Design 的 SKILL.md 适配器（packages/plugin-runtime/src/adapters/agent-skill.ts 的 adaptAgentSkill）完全兼容——旧格式的 SKILL.md 会被自动适配为 mediaTypes: ["text"]。

5.2 studio.json Schema（借鉴 open-design.json + Zod）

studio.json 是 open-design.json 的多模态扩展版本，使用 Zod Schema 验证。其核心扩展在于 od 对象中新增了 mediaTypes、mediaInputs、mediaOutputs 和 mediaPipeline 字段。StudioManifestSchema 通过 z.object().extend() 方法在 PluginManifestSchema 基础上扩展，保持了对 Open Design 插件生态的完全兼容——任何符合 open-design.json 规范的插件都可以被 Agent Studio 直接加载，因为 StudioManifestSchema 是 PluginManifestSchema 的超集。

第六章 多模态流式协议

Open Design 的 SSE 流式协议定义了 6 种事件类型（start/agent/stdout/stderr/error/end），其中 agent 事件包含 11 种子类型。所有事件都遵循 SseTransportEvent 信封格式，定义在 packages/contracts/src/sse/common.ts 中。Agent Studio 的 MultimodalSseEvent 在此基础上扩展至 12 种事件类型，新增 6 种多模态专用事件，每个事件都有严格的 Zod Schema 定义。

事件名称	类型	载荷类型	说明
------	----	------	----

start	继承	ChatSseStartPayload	运行开始
agent	继承	DaemonAgentPayload	Agent 子事件（扩展多模态子类型）
stdout/stderr	继承	ChatSseChunkPayload	原始标准输出/错误
error/end	继承	SseErrorPayload / ChatSseEndPayload	错误/结束
media:chunk	新增	MediaChunkPayload	媒体数据分片（Base64 编码）
media:progress	新增	MediaProgressPayload	媒体生成进度（百分比 + ETA）
media:complete	新增	MediaCompletePayload	单个媒体资产生成完成
media:error	新增	MediaErrorPayload	媒体生成错误（含可重试标志）
artifact:media	新增	ArtifactMediaPayload	多模态工件事件
critique:panel	新增	CritiquePanelPayload	评审剧院面板事件

表 6-1 MultimodalSseEvent 事件类型

6.1 media:chunk 帧结构（借鉴 SseTransportEvent 信封格式）

media:chunk 事件将大型媒体文件拆分为可管理的分片进行传输。每个分片包含 runId、mediaType (image/audio/video)、chunkIndex、totalChunks、encoding (base64/url/raw-pcm)、format (jpeg/png/mp3/wav/mp4/webm)、data (编码后的数据) 和 isFinal 标志。前端根据这些信息重组媒体文件：图片使用渐进式 JPEG 解码器逐分片渲染，音频使用 Web Audio API 的 PCM 缓冲区逐帧播放，视频使用 MediaSource Extensions 逐段追加。这种分片设计与 Open Design 的 SseTransportEvent 信封格式一脉相承——每个分片都是一个自描述的传输单元。

第七章 多工件流式渲染

Open Design 的 Artifact Streaming Rendering 系统由两个核心组件构成：Streaming Artifact Parser (apps/web/src/artifacts/parser.ts) ——一个实时检测 标签并提取增量 HTML 内容的状态机；Srcdoc Iframe Sandbox Renderer (apps/web/src/runtime/srcdoc.ts) ——将提取的 HTML 注入到 srcdoc iframe 沙箱中并注入运行时桥接脚本。Agent Studio 的 MultiArtifactParser 在此基础上扩展为支持四种媒体类

型的渲染器。

渲染器	媒体类型	渲染技术	核心特性
TextRender er	text	srcdoc iframe 沙箱 （与 Open Design 完全一致）	实时 HTML/CSS 渲染 选择桥接 + 快照桥接
ImageRende rer	image	Canvas API 渐进式 JPEG 解码	逐分片绘制 缩放/裁剪/滤镜预览
AudioRende rer	audio	Web Audio API PCM 缓冲区 + 波形绘制	实时音频播放 波形可视化 + 频谱分析
VideoRende rer	video	MediaSource Extensions 逐帧追加 + Canvas 预览	关键帧即时显示 播放控制 + 时间轴编辑

表 7-1 四种媒体渲染器对比

与 Open Design 类似，Agent Studio 也在每个渲染器中注入了运行时桥接脚本。TextRenderer 继承了 Open Design 的全部桥接（选择桥接、调色板桥接、快照桥接、手动编辑桥接等），这些桥接通过 postMessage 与主线程通信，遵循 Open Design 建立的 od:srcdoc-transport 协议。ImageRenderer 新增了裁剪桥接和滤镜桥接，AudioRenderer 新增了播放控制桥接和波形交互桥接，VideoRenderer 新增了时间轴桥接和帧提取桥接。所有新桥接都遵循 od:srcdoc-transport 协议，确保了架构的一致性。

第八章 多模态评审剧院

Open Design 的 Critique Theater 是其最具独创性的架构模式之一。5 个 AI 评审角色对生成的作品进行多维度评分，通过加权综合得分决定是否发布。如果综合得分低于阈值，系统自动进入下一轮迭代优化。Rolling Ratchet 渐进发布机制确保新功能从 M0 逐步发布到 M3。Agent Studio 将 Critique Theater 扩展为 Multimodal Critique，新增第 6 个评审角色——"模态专家"（modalist），专门负责评估跨模态一致性。

角色	权重	评审维度	说明
designer	0.20	视觉和谐度、构图平衡、色彩运用	继承 Open Design，权重从 0 调整
critic	0.20	创意原创性、情感冲击力、叙事连贯性	继承 Open Design，权重从 0.40 调整
brand	0.15	品牌一致性、调性匹配	继承 Open Design
ally	0.15	可访问性、对比度、字幕质量	继承 Open Design
copy	0.10	文案质量、语法正确	继承 Open Design

modalis t	0.20	跨模态一致性、风格同步、时序对齐	新增角色
--------------	------	------------------	------

表 8-1 六角色评审团及权重分配

8.1 Rolling Ratchet 按模态独立发布（借鉴 evaluateRollout）

Open Design 的 evaluateRollout() 函数根据连续达标天数决定功能发布阶段。Agent Studio 完全继承这一机制，关键扩展在于每种模态有独立的发布轨道。图片生成功能可能已经达到 M3 级别（连续 14 天 90% 发布率 + 95% 干净解析率），而视频生成功能可能仍在 M1 级别。这种按模态独立发布的设计确保了成熟度不同的功能可以按照各自的节奏发布，不会互相阻塞。isCritiqueEnabled() 函数同样按模态分别判断——当某个模态处于 M0/M1 阶段时，该模态的评审默认关闭；达到 M2 时选择加入；达到 M3 时默认开启。

第九章 Contracts 共享缝合层

Open Design 的 packages/contracts 是整个架构的缝合线。这个包使用纯 TypeScript + Zod 定义所有跨层共享的类型和 Schema，零 Node.js 运行时依赖，被 Daemon 和 Web 两个应用共同引用。它包含 SSE 事件类型、提示词组合类型、插件清单 Schema、评审配置 Schema、API 路由合约等模块。Agent Studio 的 packages/studio-contracts 完全遵循这一模式，在 contracts 基础上扩展了多模态相关的类型定义。关键原则不变：零运行时依赖、Zod Schema 校验、Daemon 和 Web 共同引用。

模块路径	职责	关键类型
sse/	多模态 SSE 事件类型	MultimodalSseEvent、MediaChunkPayload
prompts/	多模态提示词类型	ComposeMultimodalInput、MediaContext
plugins/	技能清单 Schema	StudioManifestSchema（扩展自 PluginManifestSchema）
critique/	多模态评审 Schema	MultimodalCritiqueConfigSchema
media/	媒体类型定义（新增）	MediaType、MediaAsset、MediaGeneration
brand/	品牌系统 Schema（新增）	BrandManifestSchema（扩展自 DesignSystemSchema）

表 9-1 studio-contracts 模块结构

StudioManifestSchema 通过 z.object().extend() 方法在 PluginManifestSchema 基础上新增多模态字段，保持了对 Open Design 插件生态的完全兼容。MultimodalCritiqueConfigSchema 在 CritiqueConfigSchema 基础上新增 modalist 角色和按模态独立发布轨道。由于 Zod 的 .extend() 方法允许新增字段有默认值，旧的配置文件会

被自动填充默认权重，无需手动迁移。这种向后兼容的设计确保了生态系统的平滑迁移。

第十章 Sidecar IPC 协作

Open Design 的 Sidecar IPC 机制（`packages/sidecar`）实现了 Daemon、Web、Desktop 三个应用之间的跨进程通信。它使用 JSON-over-Unix-Socket（Linux/macOS）和 Windows Named Pipe（Windows）作为传输层，通过换行符分隔的 JSON 帧协议进行请求-响应通信，默认超时 1500ms。Agent Studio 完全继承这一 IPC 机制，并新增了 `streamMediaIpc()` 函数用于大型媒体文件的二进制流传输。

`streamMediaIpc()` 的帧协议设计如下：每个帧由 4 字节长度前缀 + 1 字节类型标志（0x01=JSON 元数据帧，0x02=二进制数据帧）+ N 字节数据组成。发送方先发送一个 JSON 元数据帧（包含媒体类型、格式、总大小等信息），然后连续发送二进制数据帧，直到传输完成。这种设计既保持了 JSON 协议的可读性，又实现了二进制数据的高效传输，同时复用了 Open Design 的 `resolveAppIpcPath()` 路径解析逻辑和 `createJsonIpcServer()` 服务端框架。

第十一章 能力门控技能系统

Open Design 的 Capability-gated Plugins 机制通过 `od.capabilities` 数组声明插件所需的运行时能力，通过信任层级决定默认授予的能力范围。`resolveCapabilitiesGranted()` 函数在运行时根据技能清单和信任层级决定授予的能力集合，`requiredCapabilities()` 函数自动推导技能所需的能力。Agent Studio 在此基础上扩展了多模态能力词汇表。

能力名称	类型	默认信任层级	说明
<code>prompt:inject</code>	继承	始终授予	最基础的能力（与 Open Design 一致）
<code>fs:read / fs:write</code>	继承	受信任	文件系统读写
<code>mcp / bash / network</code>	继承	受信任	MCP/Bash/网络访问
<code>image:generate</code>	新增	受信任	图片生成
<code>image:edit</code>	新增	受信任	图片编辑
<code>audio:generate</code>	新增	受信任	音频生成
<code>video:generate</code>	新增	受信任	视频生成
<code>media:export</code>	新增	受信任	媒体导出
<code>media:transcode</code>	新增	受信任	媒体转码

表 11-1 扩展能力词汇表

requiredCapabilities() 函数新增了对多模态能力的推导逻辑：如果技能的 studio.json 中声明了 mediaOutputs 包含 audio 类型，则自动推导需要 audio:generate 能力；如果声明了 connectors 中包含 suno-connector，则自动推导需要 connector:suno-connector 能力。这种声明式的能力推导与 Open Design 的推导逻辑完全一致——只不过推导的源字段从 od.context.mcp 和 od.connectors.required 扩展到了 od.mediaOutputs 和 od.mediaInputs。

第十二章 品牌系统即 Markdown

Open Design 的 100+ DESIGN.md 文件遵循统一的 9 节结构，其中第 9 节"Agent Prompt Guide"是专为 AI Agent 设计的机器可读提示词，包含精确的 CSS 属性值和组件描述。composeSystemPrompt() 会将活跃设计系统的 DESIGN.md 注入到系统提示词中作为权威参考。Agent Studio 将 DESIGN.md 扩展为 BRAND.md，新增 3 节多模态品牌规范，注入到 composeMultimodalPrompt() 的第 12 层。

节号	名称	类型	内容概述
1-9	视觉主题到 Agent 指南	继承	与 Open Design DESIGN.md 完全一致
10	音频品牌规范	新增	品牌音色、BPM 范围、乐器偏好、情绪图谱
11	视频品牌规范	新增	运镜风格、转场偏好、色彩分级、字幕风格
12	图片品牌规范	新增	构图原则、光影风格、后期处理、视觉一致性

表 12-1 BRAND.md 12 节结构

以下是一个科技品牌音频规范的示例片段，展示了 BRAND.md 如何将抽象的品牌调性转化为 AI Agent 可执行的创作指令。这种设计直接借鉴了 Open Design 的"Agent Prompt Guide"模式——为每种媒体类型提供精确到参数级别的创作指引。

```
## 10. 音频品牌规范 ### 品牌音色 - 主音色：温暖的原声钢琴，音色偏暖 - 辅助音色：柔和的合成器 Pad，空间感强 - 禁止：电子鼓点、失真效果、金属吉他 ### BPM 范围 - 默认：72-88（中慢板，温暖且不急躁） - 快节奏：90-110 ### 情绪图谱 - 主情绪：warmth / calm / trust - 可接受：hope / gentle_energy - 禁止：urgency / aggression / coldness
```

第十三章 本地优先与 BYOK 模式

Open Design 的双执行模式是其商业策略的核心：CLI 本地模式（无需 API Key）和 API 代理模式（BYOK，用户自带 OpenAI/Anthropic/Azure/Google/Ollama 的 API Key）。两种模式共享同一套 SSE 流式协议和 Artifact 渲染管线。API 代理模式内置了 SSRF 防护，禁止请求私有 IP 地址。Agent Studio 完全继承这一双执行模式，并针对多模态场景扩展了媒体配

额管理和媒体缓存机制。

媒体配额管理通过 token-bucket 算法实时追踪每种媒体的配额消耗：图片生成按张数计费，音频生成按时长计费，视频生成按时长和分辨率计费。媒体缓存机制将生成的媒体文件缓存在 .studio/cache/ 目录中，相同参数的重复请求可以直接从缓存返回，避免重复调用 Agent CLI，显著降低本地计算资源的消耗。SSRF 防护扩展到了媒体 URL：当 Agent 请求下载外部媒体资源时，Daemon 层会先解析 URL 的 IP 地址，验证其不是私有地址后才允许下载。

第十四章 完整数据流与包结构

本章将前面所有章节的设计元素整合为一个完整的架构视图，展示数据如何在三层架构之间流动，以及各模块如何协作完成一个多模态内容创作任务。每一步都标注了对应的 Open Design 原始模式。

步骤	描述	对应的 Open Design 模式
1	用户在 StudioWeb 输入创作需求	3-Layer Architecture (Web 层)
2	StudioWeb 通过 HTTP POST 发送至 Daemon	3-Layer Architecture (Web-Daemon)
3	Daemon 从 studio-contracts 获取请求类型校验	Contracts as Architecture Seam
4	Daemon composeMultimodalPrompt() 组装 21 层提示词	Prompt Composition (扩展)
5	Daemon 根据 mediaType 选择 MultimodalAgentDef	Agent-agnostic Adapter (扩展)
6	Daemon 通过 buildArgs() 构建 CLI 参数，spawn 子进程	Agent Execution Layer
7	Agent CLI 进程执行推理，输出流式事件	Agent Execution Layer
8	Daemon 解析流式事件，转发为 MultimodalSseEvent	SSE Streaming (扩展)
9	StudioWeb 接收 SSE 事件，分发到对应渲染器	Artifact Streaming (扩展)
10	渲染器实时渲染文字/图片/音频/视频预览	MultiArtifactParser + 4 渲染器
11	创作完成后，Daemon 触发 Multimodal Critique	Critique Theater (扩展为 6 角色)
12	6 角色 AI 评审团评分，决定是否发布或迭代	Critique Theater + Rolling Ratchet
13	通过评审的资产存储到 .studio/assets/	Local-first + BYOK
14	Sidecar IPC 通知 Desktop 应用更新	Sidecar IPC

表 14-1 完整数据流步骤

14.1 Monorepo 包结构（借鉴 Open Design monorepo 组织）

包路径	职责	核心内容
apps/studio-daemon/	Express 5 + SQLite 后端	核心路由、Agent 管理、流式事件路由
apps/studio-web/	Next.js 前端	UI 交互、多模态渲染、SSE 客户端
apps/studio-desktop/	Electron 桌面应用	本地文件系统访问、原生窗口
packages/studio-contracts/	共享类型与 Schema	Zod Schema、SSE 类型、提示词类型（零依赖）
packages/studio-sidecar/	Sidecar IPC	JSON-over-Unix-Socket、媒体流传输
packages/plugin-runtime/	插件运行时	清单解析、SKILL.md 适配、合并策略
packages/agui-adapter/	AG-UI 双向适配器	事件编码/解码、协议桥接
brands/	品牌系统集成	100+ BRAND.md 多模态品牌规范
skills/	内置技能定义	文字/图片/音频/视频创作技能
plugins/_official/	官方插件	官方打包的连接器和工具插件

表 14-2 Monorepo 包结构

14.2 跨模式依赖链

Agent Studio 的跨模式依赖链与 Open Design 一脉相承。studio-contracts 定义所有 Schema 形状，StudioDaemon 和 StudioWeb 共同消费它们。plugin-runtime 使用 contracts 的 StudioManifestSchema 验证文件系统插件。Trust/Capabilities 根据清单的 studio.capabilities 在运行时门控技能能力。Prompt Composer 读取技能快照和品牌系统注入系统提示词。Agent Adapters 接收组合后的提示词并 spawn CLI 进程。SSE Protocol 从 Agent CLI 流式传输事件到 Daemon 再到 Web。MultiArtifactParser 从流中提取多模态工件，4 种渲染器在沙箱环境中实时渲染。Critique Theater 对工件质量进行 6 角色 AI 评审。Sidecar IPC 协调 Daemon、Web 和 Desktop 进程。每一个环节都直接对应 Open Design 的一个核心模式，并进行了精确的多模态扩展。